

Proyecto 3 de Taller a la Programación

Adivina en qué estoy pensando >:D

José Antonio Azofeifa Ugalde

Diego Andrés Mora Rojas

1° semestre 2026

Índice:

- Descripción general.....1
- Diseño orientado a objetos.....1
- Explicación de las clases.....1
- Manejo de archivos.....2
- Aprendizaje del árbol.....3
- Interfaz gráfica.....3
- Capturas de pantalla.....4
- Conclusiones y demás.....7

Descripción general

El proyecto consiste en un programa capaz de intentar adivinar un elemento mediante preguntas de Sí o No. Para ello se utilizan árboles binarios de decisión, donde cada nodo representa una pregunta y cada hoja representa una respuesta posible. Se permite cargar diferentes árboles en archivos .json, realizar partidas utilizando dichos árboles y aprender nuevas respuestas cuando no logra adivinar correctamente. Cuando se aprende una nueva respuesta, esta se almacena dentro del árbol y puede guardarse automáticamente para futuras ejecuciones. La parte más difícil fue tanto el crear el código para jugar el árbol, como también garantizar que la información del árbol (cuando este se modifique) pudiera almacenarse y recuperarse correctamente desde archivos externos (almacenar la pregunta con las respuestas en el nodo correspondiente), aunque también el crear las clases fue relativamente retador.

Diseño orientado a objetos

La estructura principal del programa se divide en tres clases importantes:

- `Nodo_decision`
- `Arbol`
- `Ventana`

Cada una posee una responsabilidad específica para evitar mezclar funciones y facilitar el mantenimiento del código:

- La clase `Nodo_decision` se encarga únicamente de representar cada nodo individual del árbol (pregunta, respuesta y atributos relacionados con las opciones “sí” y “no”).
- La clase `Arbol` administra la estructura completa, incluyendo carga, guardado y recorrido de nodos (`Nodo_decision` maneja los nodos de manera específica; `Arbol` se encarga del mapa completo).
- La clase `Ventana` se encarga de la interfaz gráfica o ventanas y de la interacción con el usuario.

Esta separación permite modificar una parte del proyecto sin afectar necesariamente las demás.

Explicación de las clases

Nodo_decision

Esta clase representa cada nodo dentro del árbol. Sus atributos principales son:

- `pregunta`
- `respuesta`

- sí
- no

Si un nodo posee una pregunta, entonces corresponde a una decisión intermedia. Si posee una respuesta, entonces corresponde a una hoja final del árbol. Además incluye funciones para:

- determinar si es hoja
- convertir el nodo a diccionario
- reconstruir nodos desde diccionarios

Esto facilita el almacenamiento en archivos JSON.

Arbol

Representa el árbol completo de preguntas y respuestas. Su principal responsabilidad es administrar la estructura de datos. Entre sus funciones principales se encuentran:

- cargar archivos JSON
- guardar archivos JSON
- crear árboles por defecto
- recorrer nodos
- reiniciar partidas
- gestionar el aprendizaje

También contiene mecanismos de validación para evitar errores durante la carga de archivos.

Ventana

Es la clase encargada de la interfaz gráfica construida mediante Tkinter. Su función consiste en mostrar preguntas, botones y mensajes al usuario. La ventana no almacena directamente la lógica del árbol, sino que utiliza los métodos proporcionados por la clase Arbol para realizar las operaciones correspondientes. Entre las acciones disponibles se encuentran:

- responder Sí
- responder No
- cargar árbol
- guardar árbol
- reiniciar partida
- aprender nuevas respuestas

Manejo de archivos

Para almacenar los árboles se utilizan archivos .json. La elección de este formato se debe a que resulta sencillo de leer, modificar y transportar entre diferentes equipos. Cada nodo se representa mediante diccionarios anidados que conservan la

estructura del árbol. De esta manera es posible reconstruir completamente un árbol a partir de un único archivo. Además se incorporaron validaciones para detectar:

- archivos inexistentes
- errores de lectura
- estructuras inválidas
- datos incompletos

Cuando ocurre alguno de estos problemas, el programa puede utilizar un árbol por defecto para continuar funcionando.

Aprendizaje del árbol

Una de las características principales del proyecto es la capacidad de aprendizaje. Cuando el programa no logra adivinar correctamente, solicita:

- la respuesta correcta
- una pregunta que permita diferenciar ambas respuestas
- si la nueva respuesta corresponde al Sí o al No

Con esta información se reemplaza la hoja anterior por un nuevo nodo de decisión. Esto permite que el árbol aumente progresivamente su cantidad de conocimientos. Posteriormente los cambios pueden almacenarse dentro del archivo JSON para futuras partidas.

Interfaz gráfica

La interfaz fue desarrollada utilizando Tkinter. Se buscó que fuera sencilla de utilizar y que mostrara únicamente los elementos necesarios para jugar. La mayor parte de la interacción se realiza mediante botones y cuadros de diálogo. Los botones principales permiten responder preguntas y administrar los archivos del árbol. Los cuadros de diálogo se utilizan especialmente durante el proceso de aprendizaje y para mostrar mensajes informativos al usuario.

Capturas de pantalla

Figura 1. Pantalla principal

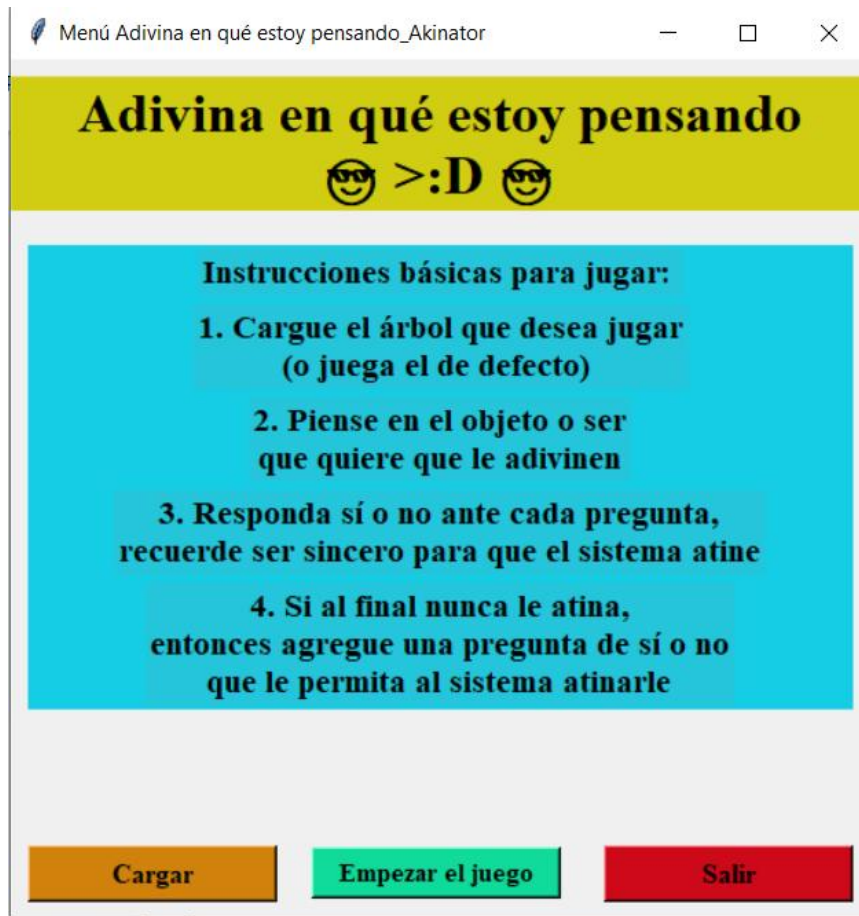


Figura 2. Árbol cargado

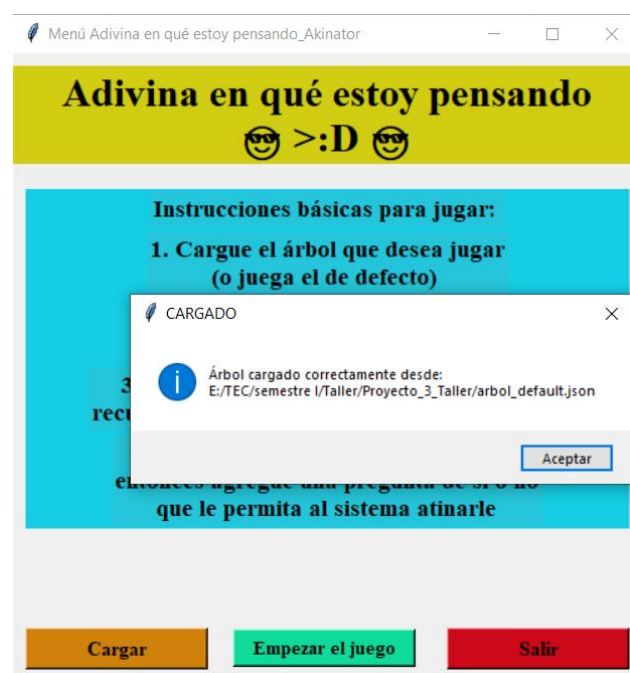


Figura 3. Preguntas en ejecución

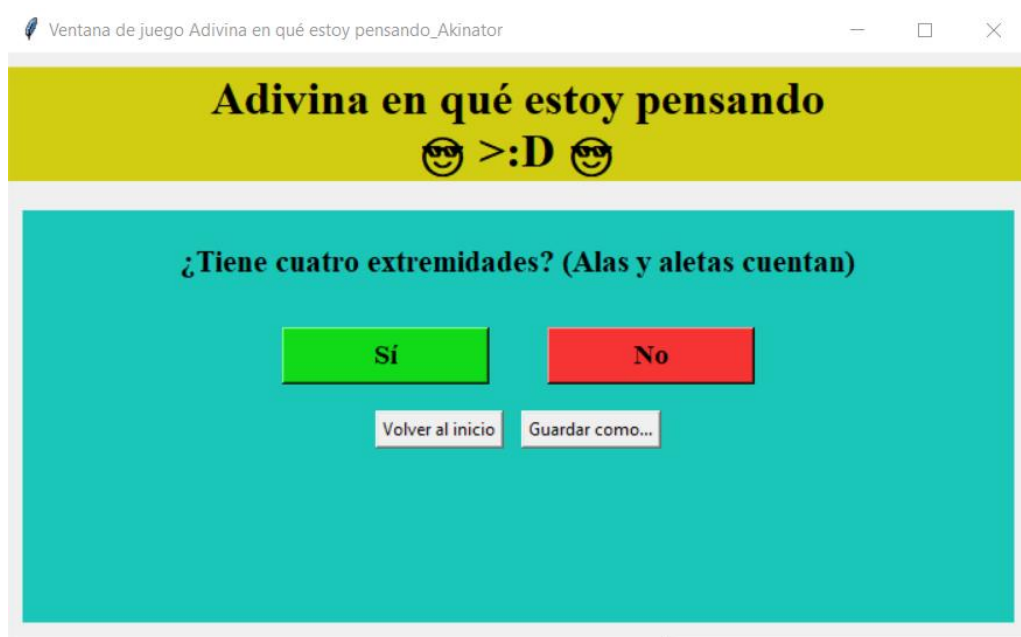


Figura 4. Respuesta correcta encontrada

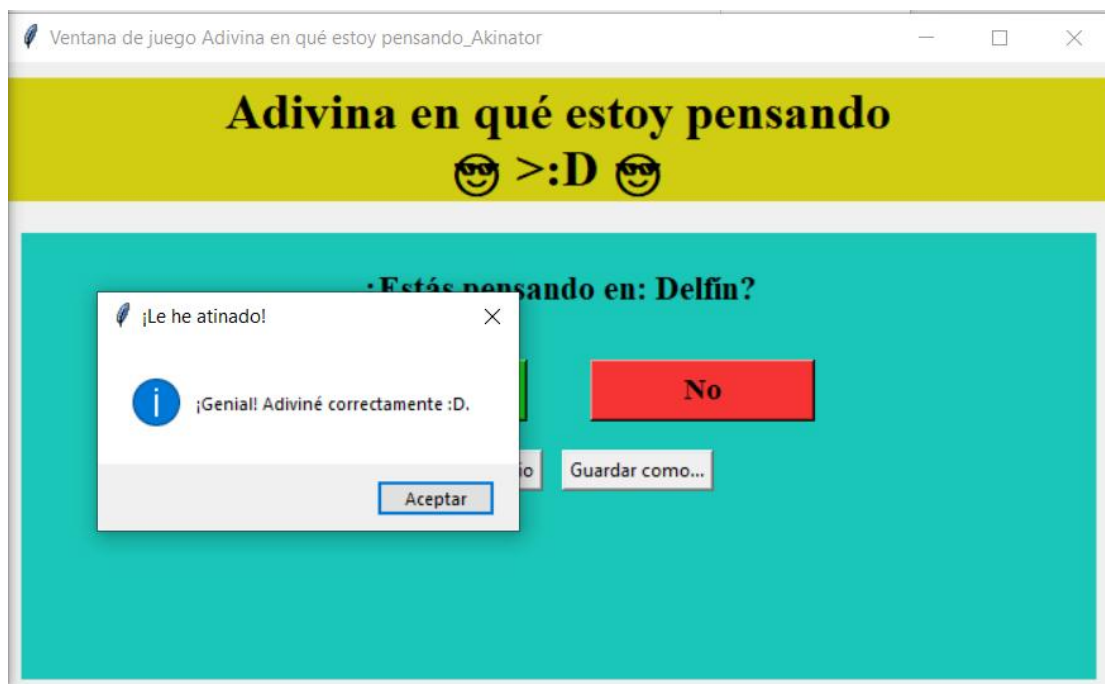


Figura 5. Proceso de aprendizaje

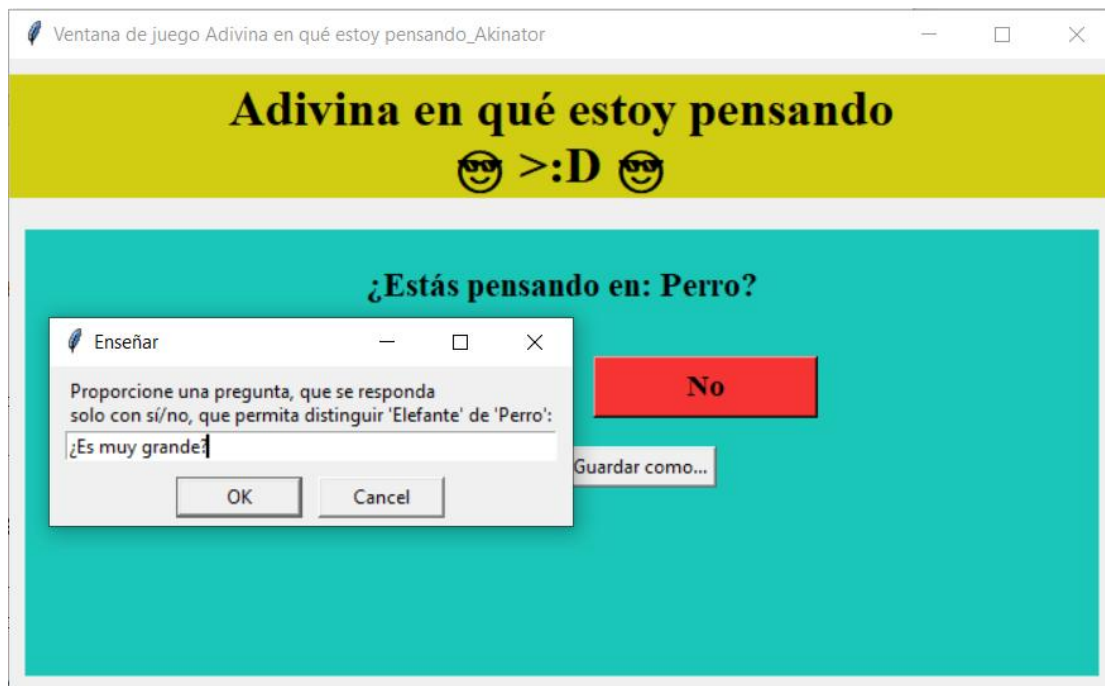


Figura 6. Guardado de árbol parte 1

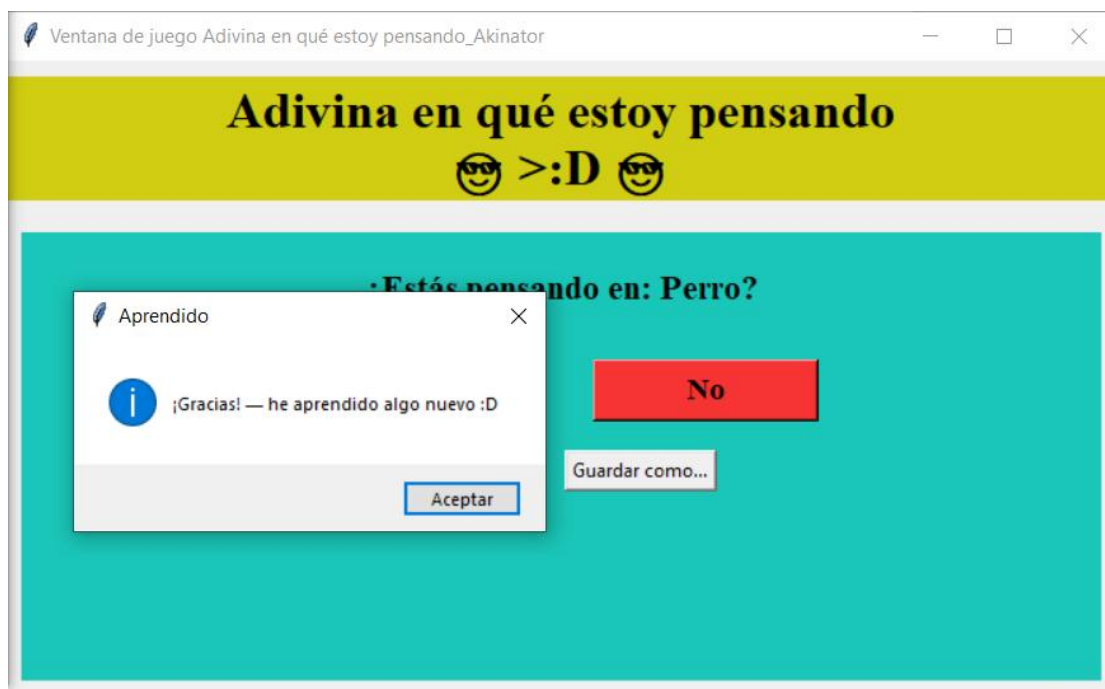


Figura 7. Guardado de árbol parte 2

```
{
  "Pregunta": "¿Tiene cuatro extremidades? (Alas y aletas cuentan)",
  "si": {
    "Pregunta": "¿Camina principalmente?",
    "si": {
      "Pregunta": "¿Es mamífero?",
      "si": {
        "Pregunta": "¿Es muy grande?",
        "si": {
          "Respuesta": "Elefante"
        },
        "no": {
          "Respuesta": "Perro"
        }
      },
      "no": {
        "Respuesta": "Salamandra"
      }
    },
    "no": {
      "Respuesta": "Salamandra"
    }
  },
  "no": {
    "Respuesta": "Salamandra"
  }
},
},
```

Conclusiones y demás

Se logró implementar correctamente el sistema en árboles binarios, que simultáneamente se modificaba al aprender algo nuevo. Fue complicado en gran parte del proceso, mas sí se logró completar en su totalidad. Desde creando las clases, verificando que los términos y nombres de atributos y variables coincidan (hubo el caso del atributo para_diccio() de la clase Nodo_desicion, que en una de las veces que lo utilicé escribí pra_diccio(), y eso obviamente hacía que el código no se ejecutara correctamente; y la parte más fea de este problema fue que me costó encontrarlo (tuve que pedirle a ChatGPT que revisara el código y me dijera dónde estaba ese error)), hasta luego creando las verificaciones o filtros para los archivos, que durante el proceso de juego las ramas sí se muestren y sirvan, y finalmente comprobar que cuando aprende, agregue el nodo al mapa y que este no se vuelva en blanco más bien (muchas veces el mapa estaba completo y bien, pero cuando se probaba, se vaciaba completamente y así se quedaba aunque se cierre el juego completo). No obstante, a pesar de todas estas trabas y dificultades, el juego funciona en su totalidad y sin errores apreciables o que impidan el correcto funcionamiento del mismo, ni afecten la experiencia de juego.